

Quiz #5

CS 225 Data Structures

Spring 2026

7:00-8:00pm, Thursday, April 23

Name	Hanbin Zhang
ZJU ID	
Lab TA	

- This is a **closed book** and **closed notes** exam. No electronic aids are allowed, either.
- You should have 4 problems total on 6 pages. You must write on the exam.
- Unless otherwise stated in a problem, assume the best possible design of a particular implementation is being used.
- Unless the problem specifically says otherwise, (1) assume the code compiles, and thus any compiler error is an exam typo (though hopefully there are not any typos), and (2) assume you are NOT allowed to write any helper methods to help solve the problem, nor are you allowed to use additional arrays, lists, or other collection data structures unless we have said you can.
- Please put your name at the top of each page.
- Please fill in your **Lab Section** or your **Lab TA's name** to help us organize and grade your papers.
- Don't panic and good luck!

Problem 1 _____/12
Problem 2 _____/10
Problem 3 _____/8
Problem 4 _____/10

Total _____/40



$$(m-1) (1 + m + m^2 + \dots + m^h)$$

$$= (m-1) \frac{m^{h+1} - 1}{m - 1} = m^{h+1} - 1$$

$$(1 + (t-1) (2 + 2t + \dots + 2t^{h-1})) = 2(t-1) \frac{t^h - 1}{t - 1} + 1 = 2t^h - 1$$

Problem 1 (12 points):

1. In a B-tree of order m , fill in the following:

Maximum number of children per node: m

Maximum number of keys per node: $m-1$

Minimum number of children (non-root): $\lceil m/2 \rceil$

Minimum number of keys (non-root): $\lceil m/2 \rceil - 1$

2. A B-tree node contains 4 keys

$k \leq$

a) How many children must this node have?

5 children

b) Is it possible for this node to have 6 children? (Yes / No)

3. Consider a B-tree of order 3 and height 2 (root at height 0).

What is the **maximum number of keys** that can be stored?

$$m = 3, h = 2$$

$$3^{(2+1)} - 1 = 3^3 - 1 = 27 - 1 = 26$$

4. A KD-tree is built by inserting the following 2D points in order:

(5,4), (2,3), (8,7), (6,1), (7,2)

Assume alternating split dimensions: $x \rightarrow y \rightarrow x \rightarrow y \rightarrow \dots$

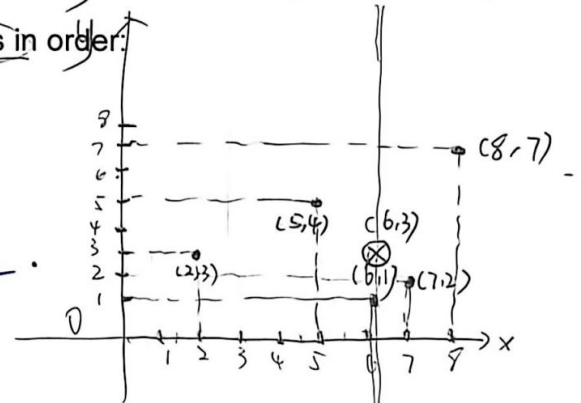
a) Fill in the tree structure:

Root: (6,1)

Left child of root: (5,4)

Right child of root: (8,7)

Left child of (8,7): (7,2)



b) For query point (6,3), list the nodes visited **in order** during nearest neighbor search:

(6,1) $\rightarrow$ (8,7) $\rightarrow$ (7,2) $\rightarrow$

5. In KD-tree nearest neighbor search, pruning happens when:

- A. Distance to splitting plane < current best distance
- B. Distance to splitting plane > current best distance
- C. The node has two children
- D. The query lies exactly on the splitting plane

剪枝

大了不用看

英语不好

是一个个 insert,
不是直接 sort, 然后
原地建!

$\sqrt{2}$



Problem 2 (10 points):

4 1 2

1. Consider the following hash-function for 4-digit integer keys (which means the key is in range 1000-9999) x and a table of size 10:

$$\text{hash}(x) = \text{floor}(x / 1000)$$

0 ~ 9

In no more than 20 words, explain why this is NOT a good general-purpose hash function.

2. Consider a hash table of size 11, with starting index 0, and the insertion sequence 0, 1, 2, 3, 4, 5, 6, 7, 12, 23, 34, 45, 56, 67, 78. Assume the table is initially empty. 8, 9, 10, 11

a) Using linear probing with $h(x) = x \% 11$, fill in the final contents of the table. Write "NUL" for an empty location.

[0] _____	[1] <u>12</u>	[2] <u>23</u>	[3] <u>34</u>	[4] <u>45</u>	[5] <u>56</u>
[6] <u>67</u>	[7] <u>78</u>	[8] _____	[9] _____	[10] _____	

b) Using double hashing with $h_1(x) = x \% 11$ and $h_2(x) = 7 - (x \% 7)$, fill in the final contents of the table. Write "NUL" for an empty location.

[0] _____	[1] <u>12</u>	[2] <u>34</u>	[3] _____	[4] <u>67</u>	[5] <u>45</u>
[6] <u>23</u>	[7] <u>78</u>	[8] <u>56</u>	[9] _____	[10] _____	

$$h(x) = h_1(x) + k \cdot h_2(x)$$

1, 2, 3

3. Under a good hash function defined in class, for separate chaining with load factor $\alpha = 1.6$, fill in the expected running time of find.

Successful find: _____

Unsuccessful find: _____

4. In probe-based hashing, when the load factor becomes too large, describe what rehashing does. Your answer must mention both the new table and the old keys. Use no more than 30 words.



Problem 3 (8 points): Dictionaries

1. In a graduate program, a faculty advisor may have many students, but a student will strictly have only one faculty advisor. University Office wants to quickly determine the advisor of a student, given the student's university ID. Which dictionary representation is most appropriate?

- ☒ a) Keys: advisor names; Values: lists of student IDs
- ☒ b) Keys: student IDs; Values: advisor names
- ☒ c) Keys: concatenated strings of studentID + advisorName; Values: boolean
- ☒ d) Keys: unique integers 0,1,2,...; Values: pairs of (studentID, advisorName)

2. Which of the following are valid operations covered in lecture of the Dictionary ADT? Select all that apply.

- ☒ a) remove the first item in the dictionary X 不一定有!
- ☒ b) overloaded operator [], to facilitate the usage of find function in dictionary ADT
- ☒ c) given a value, return the key of the dictionary entry with the given value
- ☒ d) given a key, remove the entry that contains the key
- ☒ e) insert a new key-value pair into the dictionary
- ☒ f) given a value, find the set of keys mapped to that value

3. Which of the following data-structures can be used to implement a Dictionary so that all of its core functions (insert, lookup, delete) have a worst case running time strictly better than $O(n)$? Select all that apply.

- ☒ a) Linked list
- ☒ b) Binary search tree
- ☒ c) AVL-tree
- ☒ d) BTree
- ☒ e) Unsorted array

即使退化为 vector 也不是 $O(n)$, 是 $O(\log(n))$

一般为 $O(\log_m n)$

4. Which of the following collection of function signatures corresponds to the Dictionary ADT? Select all that apply.

- ☒ a) void insert(key, value); value remove(key); key* find(value);



- ~~b) void insert(key, value); void remove(key, value); void find(key);~~
~~c) value insert(key, value); value remove(key); value find(key);~~
~~d) void insert(key, value); void remove(key); value find(key);~~
 e) None of the other items describe a dictionary.

Problem 4 (10 points): lab_btree

You are asked to implement the **private recursive helper function** `insert(BTreeNode* subroot, const DataPair& pair)` used to insert a key-value pair into a BTree. All other relevant parts of the BTree have been implemented for you, including the public `insert(const K& key, const V& value)` function and the `split_child(BTreeNode* parent, size_t child_idx)` helper, and the `insertion_idx(const std::vector& elements, const DataPair& pair)` helper. The `insertion_idx` function is used to find the insertion index of a given element into a given sorted vector. The declaration is shown here:

```
size_t insertion_idx(const std::vector<DataPair>& elements,
                    const DataPair& pair) const;
```

Suppose you have the following code:

```
template <class K, class V>
class BTree {
private:
    struct DataPair {
        K key;
        V value;
        bool operator==(const DataPair& rhs) const {
            return this->key == rhs.key; }
    };
    struct BTreeNode {
        std::vector<DataPair> elements;
        std::vector<BTreeNode*> children;
        bool is_leaf;
    };
    unsigned int order;
    BTreeNode *root;
    void split_child(BTreeNode* parent, size_t child_idx);
    void insert(BTreeNode* subroot, const DataPair& pair);

public:
    void insert(const K& key, const V& value);
};

// std::vector reminder:
// v.size()           -> number of elements
// v[i]               -> element at index i
// v.begin()          -> iterator to first element
// v.end()             -> iterator to one past last element
// v.insert(pos, x)    -> insert x before pos
// v.begin() + i       -> iterator pointing to index i
```



```
template <class K, class V>
void BTree<K, V>::insert(BTreeNode* subroot, const DataPair& pair)
{
    if(subroot == nullptr) return;
    size_t first_larger_idx = insertion_idx(subroot->elements, pair);
    // TODO: Your code goes here
}
```

